

面向 LLM / Agent Systems 科研的深度学习零基础课

第 05 讲

实验设计：从 $xW + b$ 到 PyTorch `nn.Linear`

实验目标：通过控制变量和数值对照，验证 Weight、Bias、Batch Broadcasting、参数数量以及 PyTorch Linear 的真实计算。

实验设计文档

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

1 实验定位

本实验仍然不训练模型，而是回答一个更底层的问题：

Linear Layer 的每个数字到底从哪里来？

实验结构：

问题 → 预测 → 运行 → 观测 → 解释 → 结论

2 实验环境

- Python 3.x；
- NumPy；
- PyTorch（用于最后独立核对）；
- CPU 即可；
- 代码：lesson05_linear_weight_bias_experiment.py。

运行：

```
python lesson05_linear_weight_bias_experiment.py
```

3 实验 1：手算 $xW + b$ 并与 NumPy 对照

研究问题

一个 $3 \rightarrow 2$ 的 Linear Layer，两个输出是否分别等于“输入 dot Weight 对应列 + 对应 Bias”？

实验数据

$$x = [2, -1, 3]$$
$$W = \begin{bmatrix} 0.5 & -1.0 \\ 2.0 & 0 \\ -0.5 & 1.5 \end{bmatrix}, \quad b = [0.2, -0.3]$$

预测

$$xW = [-2.5, 2.5]$$

$$y = [-2.3, 2.2]$$

验证标准

手算结果与 $\text{NumPy } x @ W + b$ 应完全一致。

4 实验 2: Batch 共享同一组参数

定义 3 个样本:

$$X : (3, 3)$$

仍使用相同:

$$W : (3, 2), \quad b : (2,)$$

预测:

$$Y : (3, 2)$$

关键观察

1. 第一行输出是否与实验 1 完全一致?
2. 是否所有样本都使用同一个 W ?
3. Bias 是否被自动加到每一行?

5 实验 3: 只改变 Bias

控制变量设计

保持 X 、 W 完全不变, 只把:

$$b = [0.2, -0.3]$$

改成:

$$b' = [10.2, -5.3]$$

Bias 差值:

$$\Delta b = [10, -5]$$

假设

所有样本第 1 个输出都应增加 10; 第 2 个输出都应减少 5。

科研视角

这是一个最简单的控制变量实验: 只修改一个组件, 观察输出的系统性变化。以后做 Ablation Study, 本质思路仍然是“尽量控制其他条件, 只改变研究因素”。

6 实验 4：只改变 Weight 的一列

复制 W 得到 W' ，只把第 0 列乘 2。

假设

由于第 0 列专门参与第 0 个输出，所以第 0 个输出会改变；第 1 列没有变，因此第 1 个输出保持不变。

为什么这个实验重要

它把“Weight 每一列对应一个输出通道”从语言描述变成可观察现象。

7 实验 5：参数数量核对

对于 $3 \rightarrow 2$ ：

$$\#W = 3 \times 2 = 6$$

$$\#b = 2$$

$$\#total = 8$$

代码直接打印三项，作为手算核对。

8 实验 6：与 PyTorch nn.Linear 对照

研究问题

为什么课程 NumPy 中 $W : (3, 2)$ ，而：

```
nn.Linear(3, 2).weight.shape
```

却是：

$(2, 3)$

实验方法

把 NumPy 的 W^T 复制到 PyTorch：

```
layer.weight.copy_(torch.tensor(W.T))  
layer.bias.copy_(torch.tensor(b))
```

再让同一个 Batch 通过 `layer(X)`。

假设

PyTorch 输出应与 NumPy:

$$XW + b$$

完全一致。

实验意义

如果结果一致，就说明两种 Shape 约定只是存储方向不同，并没有数学矛盾。

9 实际运行结果

本材料包中的脚本已经执行，得到：

- 单样本： $y = [-2.3, 2.2]$ ；
- Batch 输出 Shape：(3, 2)；
- 更改 Bias 后，所有样本差值均为 [10, -5]；
- 只修改 Weight 第 0 列后，第 1 个输出通道保持不变；
- 参数数：Weight 6 + Bias 2 = 8；
- PyTorch `weight.shape=(2,3)`；
- PyTorch 输出与 NumPy 完全一致。

完整控制台结果见 第 05 讲 _ 实验实际输出.txt。

10 实验记录表

实验	我的预测	实际结果	解释
实验 1 单样本			
实验 2 Batch			
实验 3 改 Bias			
实验 4 改 Weight			
实验 5 参数数			
实验 6 PyTorch			

11 实验后必须回答的 6 个问题

1. 为什么 $3 \rightarrow 2$ 的 Weight 在课程约定中是 $(3, 2)$?
2. 为什么 Bias 只有 2 个数?
3. 为什么一个 Batch 中所有样本共享同一个 Weight?
4. 只改变 Bias 与只改变 Weight, 输出变化有什么不同?
5. 为什么 PyTorch 的 Weight Shape 看起来“反了”?
6. $Q = XW_Q$ 与本实验有什么本质区别?

本节检查点

最低验收: 不运行代码时, 你能手算实验 1; 看到 `nn.Linear(3, 2)` 时, 你能预测 PyTorch weight 为 $(2, 3)$ 、bias 为 $(2,)$ 、输入 $(B, 3)$ 输出 $(B, 2)$ 。